DAA LAB EXPERIMENT: TSP Using Exhaustive Search

1. Aim

To implement the Travelling Salesman Problem (TSP) using the Exhaustive Search technique in C and analyze its time complexity.

2. Theory

The Travelling Salesman Problem (TSP) is a well-known optimization problem in which a salesperson must visit every city exactly once, return to the starting city, and minimize the total travel distance.

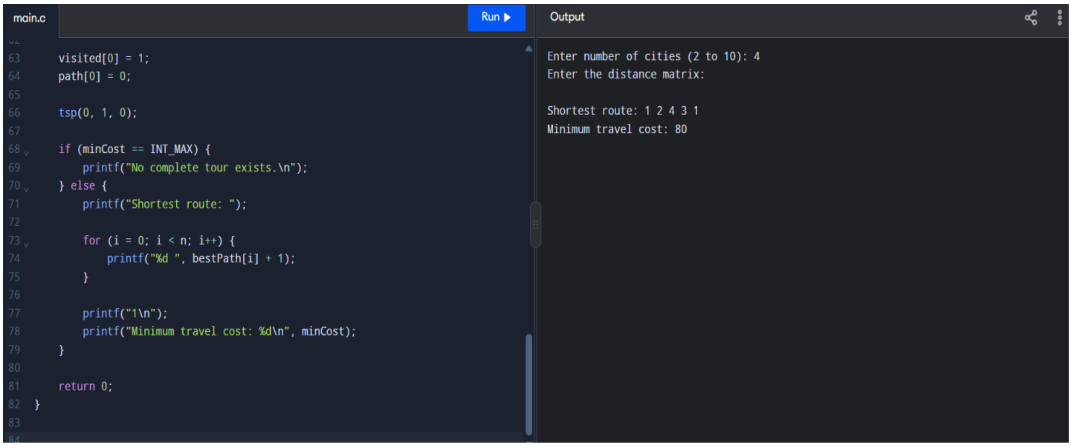
Exhaustive Search is a technique that generates and examines all possible routes to find the shortest tour. The starting city is fixed, and the algorithm considers every possible ordering of the remaining cities. It calculates the total distance of each route, including the return to the starting city, and selects the route with the minimum distance.

This method guarantees the optimal solution when all possible tours are evaluated. However, its execution time increases rapidly as the number of cities increases because the number of possible tours grows factorially.

3. Algorithm

1. Start.
2. Read the number of cities and the distance matrix.
3. Fix the first city as the starting city.
4. Generate all possible permutations of the remaining cities.
5. Calculate the total distance of each route, including the return to the starting city.
6. Compare each route's distance with the minimum distance found so far.
7. Store the route with the smallest distance.
8. Display the shortest route and its total distance

9. Stop.



The route is City 1 → City 2 → City 4 → City 3 → City 1, with a total distance of 80.

7. Complexity Analysis

Let \(n\) be the number of cities.

Time complexity: \(O(n!)\) for exhaustive enumeration of all tours when the starting city is fixed. Calculating the cost of each tour adds up to \(O(n)\) work per tour, so a more precise bound for this implementation is \(O(n \cdot (n-1)!)\), which is equivalent to \(O(n!)\).

Space complexity: \(O(n)\) auxiliary space for the visited array, current path, best path, and recursion stack. The distance matrix requires \(O(n^2)\) storage.

- Best case: Factorial growth in the number of cities.
- Average case: Factorial growth.
- Worst case: Factorial growth.

The algorithm examines all possible tours, so the order of the distances does not change its asymptotic complexity.

8. Result

The Travelling Salesman Problem was implemented successfully using Exhaustive Search in C. The program identifies the minimum-cost tour by examining all possible routes.